

Отчёт по задаче «Города и дороги»

Автор: Речкин Вадим Сергеевич

Группа: 23Б08-мм

GitHub: github.com/VadosikRRR/.../townAndRoads.cpp

1. Формулировка задачи

Есть связный неориентированный граф: вершины — города, рёбра — дороги, у каждой дороги задана длина.

Даны k столиц (по одной на каждое государство). Нужно распределить все города между государствами по правилу:

- государства обходятся по очереди от 1 до k ;
- в ход государства ему добавляется ближайший незанятый город, который имеет прямую дорогу к какому-либо уже принадлежащему этому государству городу;
- процесс продолжается, пока не будут распределены все города.

Входные данные

- n, m — число городов и дорог;
- m строк вида $u\ v\ len$;
- k — число государств;
- k чисел — номера столиц.

Выходные данные

Для каждого государства вывести его номер и список городов, которые к нему отнесены.

2. Идея решения

Поддерживаем:

- список городов каждого государства;
- массив владельца $owner[v]$ (какому государству принадлежит город v).

Далее выполняем раунды. В каждом раунде по очереди для каждого государства:

1. просматриваем все его города;

2. по их рёбрам ищем незанятые соседние города;
3. выбираем кандидат с минимальной длиной дороги;
4. при равенстве расстояний выбираем город с меньшим номером;
5. добавляем этот город государству.

Такой процесс повторяется, пока есть незанятые города.

3. Код программы

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ifstream fin("example_input.txt");

    int n, m;
    fin >> n >> m;

    vector<vector<pair<int, int>>> g(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v, len;
        fin >> u >> v >> len;
        g[u].push_back({v, len});
        g[v].push_back({u, len});
    }

    int k;
    fin >> k;
    vector<int> capitals(k + 1);
    for (int i = 1; i <= k; ++i) {
        fin >> capitals[i];
    }

    vector<int> owner(n + 1, 0);
    vector<vector<int>> states(k + 1);

    for (int s = 1; s <= k; ++s) {
        int c = capitals[s];
        owner[c] = s;
        states[s].push_back(c);
    }

    int assigned = 0;
    for (int v = 1; v <= n; ++v) {
        if (owner[v] != 0) assigned++;
    }
}
```

```

while (assigned < n) {
    bool progress = false;

    for (int s = 1; s <= k; ++s) {
        int bestCity = -1;
        int bestDist = INT_MAX;

        for (int city : states[s]) {
            for (const auto &edge : g[city]) {
                int to = edge.first;
                int len = edge.second;
                if (owner[to] != 0) continue;

                if (len < bestDist || (len == bestDist && to <
                    bestCity)) {
                    bestDist = len;
                    bestCity = to;
                }
            }
        }

        if (bestCity != -1) {
            owner[bestCity] = s;
            states[s].push_back(bestCity);
            assigned++;
            progress = true;
        }
    }

    if (!progress) {
        break;
    }
}

for (int s = 1; s <= k; ++s) {
    sort(states[s].begin(), states[s].end());
    cout << "State " << s << ":";
    for (int city : states[s]) {
        cout << ' ' << city;
    }
    cout << '\n';
}

return 0;
}

```

4. Оценка сложности

Пусть n — число городов, m — число дорог.

Каждый раунд делает просмотр рёбер, инцидентных уже захваченным городам. В худшем случае получаем порядок:

$$O(n \cdot m)$$

по времени.

Память:

- список смежности — $O(n + m)$,
- массив владельцев и списки государств — $O(n)$.

Итого по памяти:

$$O(n + m).$$

5. Пример

Вход:

```
8 10
1 2 3
1 3 2
2 4 4
3 4 1
3 5 5
4 6 2
5 6 3
5 7 4
6 8 2
7 8 1
3
1 5 8
```

Возможный вывод:

State 1: 1 2 3 4

State 2: 5 6

State 3: 7 8

Иллюстрация графа:

